# Ενότητα 05:
# The Processor:
# Datapath and Control

## ΕΕΙ 233: ΟΡΓΑΝΩΣΗ ΥΠΟΛΟΓΙΣΤΩΝ

**ΑΝΔΡΕΑΣ ΔΙΑΒΑΣΤΟΣ**

andreas.diavastos@cut.ac.cy

Τεχνολογικό Πανεπιστήμιο Κύπρου | Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

CARL

1

Τεχνολογικό Πανεπιστήμιο Κύπρου | Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

## Outline

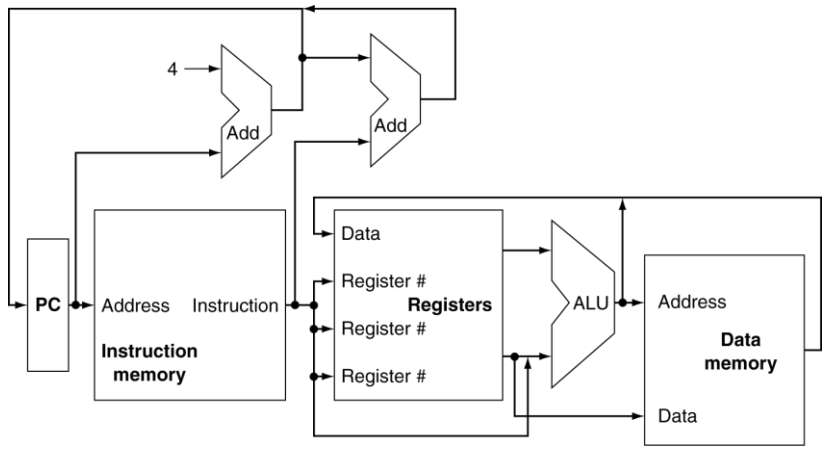
- Combinational Logic
- Abstract Σχεδιασμός MIPS CPU
- Logic Design Conventions/Basics
- Building a Datapath
- Simple Implementation (Single-cycle)
- Multi-cycle Implementation

- Υλικό από Κεφάλαιο 5 του βιβλίου του μαθήματος



2

# Υλοποίηση MIPS CPU (Abstract Overview)

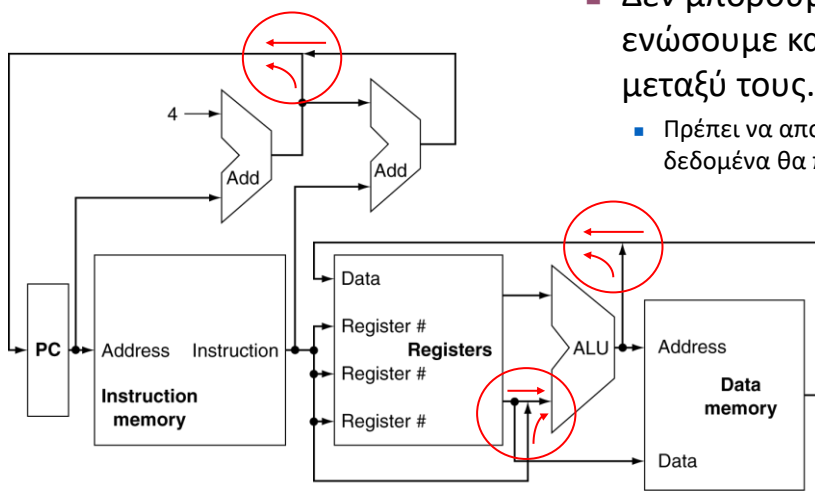






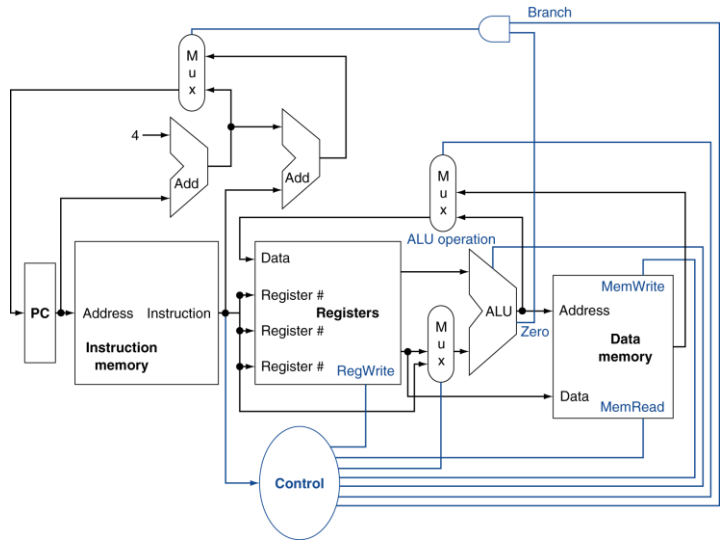



# Multiplexers




- Δεν μπορούμε απλά να ενώσουμε καλώδια μεταξύ τους…
  - Πρέπει να αποφασίσουμε ποια δεδομένα θα περάσουν

# Υλοποίηση MIPS CPU w. Control + Multiplexers



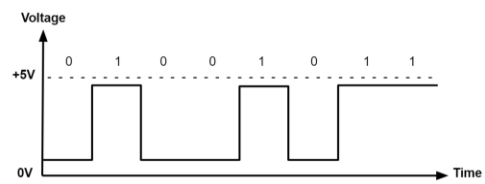







# Logic Design Basics

- Κωδικοποίηση πληροφορίας σε δυαδικό (binary)
  - Low Voltage = (bit) 0, High Voltage = (bit) 1
  - 1 wire για κάθε bit πληροφορίας που μεταφέρουμε
  - Multi-bit δεδομένα κωδικοποιούνται με multi-wire busses
  - $P \propto V^2 \times f$
    - Μειώνοντας το Voltage μειώνουμε σημαντικά την κατανάλωση ενέργειας
      - Αλλά υπάρχει όριο, διαφορετικά δεν θα μπορούμε να καταλάβουμε το 0 από το 1



# Βασικά δομικά στοιχεία - Gates

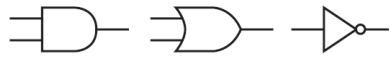


**FIGURE B.2.1 Standard drawing for an AND gate, OR gate, and an inverter, shown from left to right.** The signals to the left of each symbol are the inputs, while the output appears on the right. The AND and OR gates both have two inputs. Inverters have a single input.




| A | B | Q |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

Universal Gates




| A | B | Q |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |



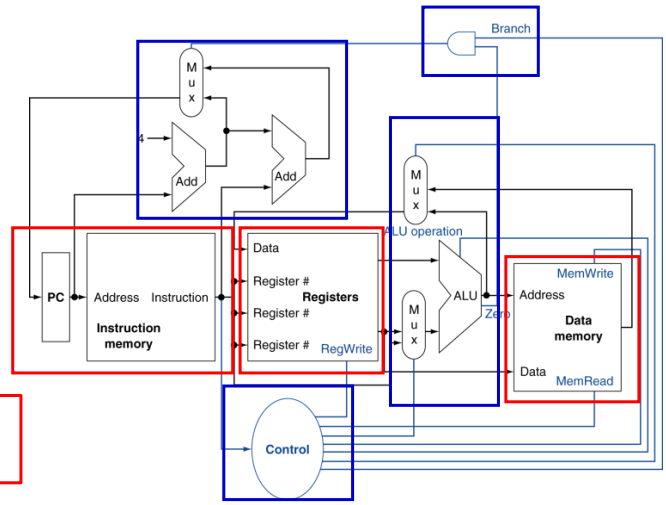




# Κατηγορίες λειτουργικών μονάδων

- Combinational elements
  - Ενεργούν πάνω σε δεδομένα
  - Δέχονται τα δεδομένα σαν είσοδο
  - Η έξοδος τους είναι συνάρτηση της εισόδου
    - Δηλαδή, εξαρτάται από την τρέχων είσοδο
  - Δεδομένου των ίδιων τιμών εισόδου, οι τιμές εξόδου είναι πάντα οι ίδιες
- State (Sequential) elements
  - Αποθηκεύουν πληροφορία
  - Κρατούν μια κατάσταση της πληροφορίας
  - Έχουν τουλάχιστον 2 εισόδους (δεδομένα και το ρολόι/clock) και 1 έξοδο (για δεδομένα)

# Κατηγορίες λειτουργικών μονάδων




Combinational Elements

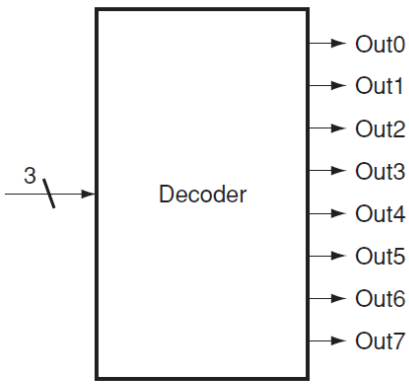
State Elements







# Combinational Elements - Decoders



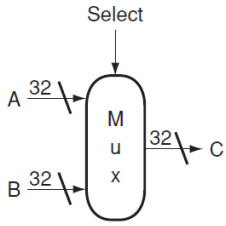

a. A 3-bit decoder

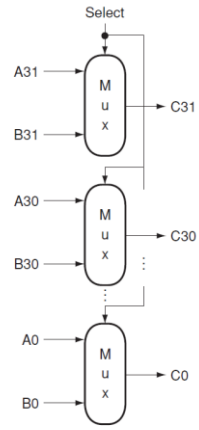
| Inputs | | | Outputs | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| 12 | 11 | 10 | Out7 | Out6 | Out5 | Out4 | Out3 | Out2 | Out1 | Out0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

b. The truth table for a 3-bit decoder

# Combinational Elements – Multiplexer

- a.k.a. Data Selector
- `C = Select ? A : B`




b. The 32-bit wide multiplexor is actually an array of 32 1-bit multiplexors



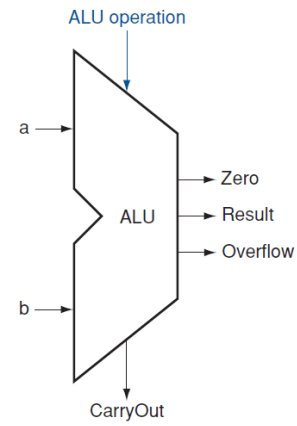




# Combinational Elements - ALU

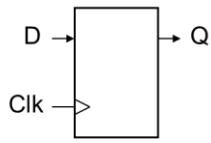
- Arithmetic and Logic Unit
- Αριθμητικές και Λογικές πράξεις

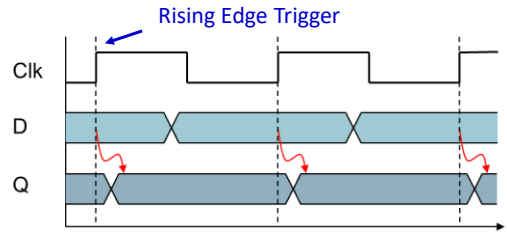
| ALU control lines | Function |
|---|---|
| 0000 | AND |
| 0001 | OR |
| 0010 | add |
| 0110 | subtract |
| 0111 | set on less than |
| 1100 | NOR |



# State Elements

- Καταχωρητές (Registers)
  - Αποθηκεύουν πληροφορία σε ένα κύκλωμα
  - Χρησιμοποιούν τον κύκλο του ρολογιού (Clk) για να ξέρουν πότε να κάνουν update την αποθηκευμένη τιμή
  - Rising-Edge-triggered (Flip-Flops): update Q όταν το Clk αλλάζει από 0 σε 1
  - Χρήσιμο για συγχρονισμό του συστήματος (read/write)



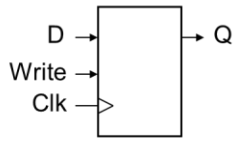


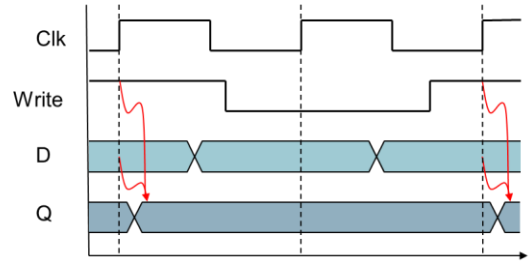





# State Elements

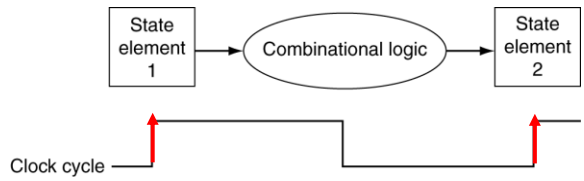
- Καταχωρητές (Registers) με έλεγχο για Write
  - Κάνουν update την αποθηκευμένη τιμή όταν το Clk αλλάζει από 0 σε 1 (Rising-Edge-triggered) ΚΑΙ το Write Control είναι 1



# Μεθοδολογία Ρολογιού (για συγχρονισμό)

- Η μεθοδολογία του ρολογιού καθορίζει πότε τα διάφορα σήματα μπορούν να διαβαστούν ή να αποθηκευτούν
  - Μεταξύ κύκλου ρολογιού
- Καθορίζει πότε τα δεδομένα είναι έγκυρα και έχουν σταθεροποιηθεί
  - Δεν θέλουμε να γράφουμε σε ένα καταχωρητή κατά τη διάρκεια της ανάγνωσης
- Είσοδος από State Element, Έξοδος σε State Element!!
- Η μεγαλύτερη καθυστέρηση, καθορίζει την περίοδο ενός κύκλου μηχανής



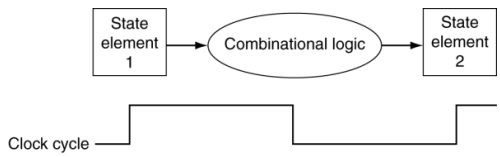


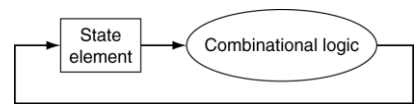





# Παράδειγμα εκτέλεσης σε 1 κύκλο ρολογιού

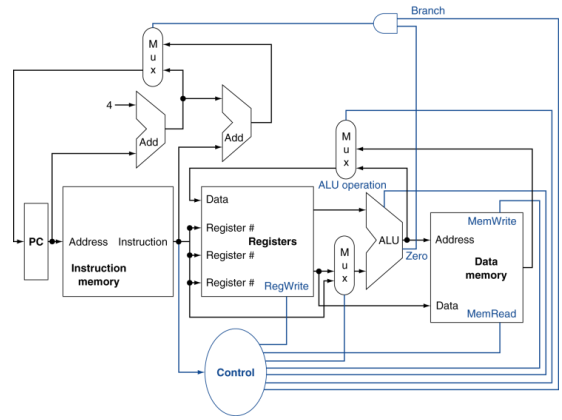
1. Ανάγνωση περιεχομένων κάποιων στοιχείων του καταχωρητή
2. Αποστολή μέσω Combinational Element/Logic
3. Γράψιμο/Αποθήκευση αποτελεσμάτων σε κάποιο στοιχείο μνήμης/καταχωρητή



# Υλοποίηση MIPS με ένα κύκλο ρολογιού

- Σε αυτή την υλοποίηση, κάθε εντολή ξεκινά και εκτελείται στην ακμή του ρολογιού (clock edge) και ολοκληρώνει την εκτέλεση της στην επόμενη ακμή του ρολογιού
- Κάθε εντολή απαιτεί 1 κύκλο ρολογιού για να εκτελεστεί
  - Μεγαλύτερη περίοδος κύκλου ρολογιού
- Απλή υλοποίηση αλλά δεν είναι πρακτικό από πλευράς επίδοσης
  - Μπορώ να εκτελέσω μόνον 1 εντολή ανά κύκλο







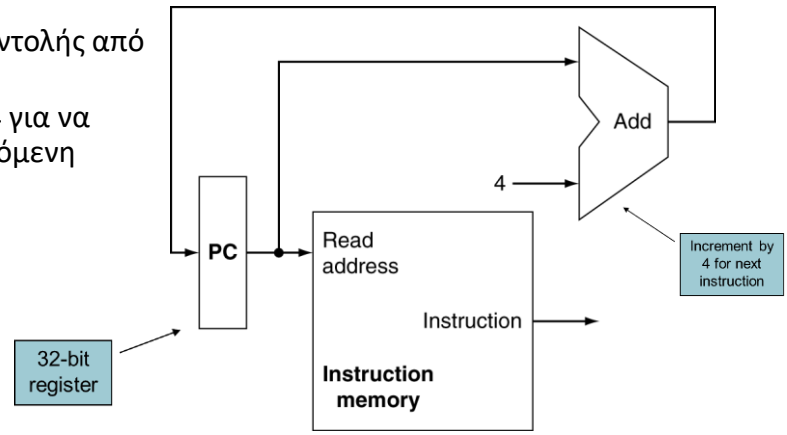



# Σχεδιασμός Διαδρόμου Δεδομένων (Datapath)

- Datapath:
  - Ακολουθία μονάδων του CPU που επεξεργάζεται δεδομένα και διευθύνσεις
    - Περιλαμβάνει: Registers, ALUs, MUXs, Memories, etc.
- Απαιτούνται διαφορετικές μονάδες (Datapaths) ανά κατηγορία εντολών
  - Διαχείριση του Program Counter (PC)
  - Εντολές τύπου R (Αριθμητικές εντολές)
  - Εντολές μνήμης τύπου I (Immediate)
  - Εντολές διακλάδωσης τύπου I και J

# Instruction Fetch: Διαχείριση PC

- Βήματα:
  1. Προσκόμιση (Fetch) εντολής από τη μνήμη
  2. Αύξηση του PC κατά 4 για να ετοιμαστεί για την επόμενη εντολή





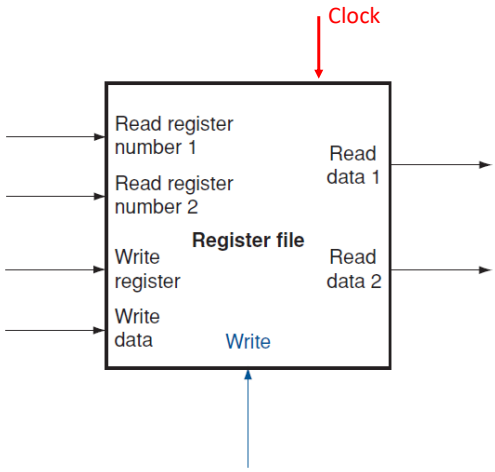


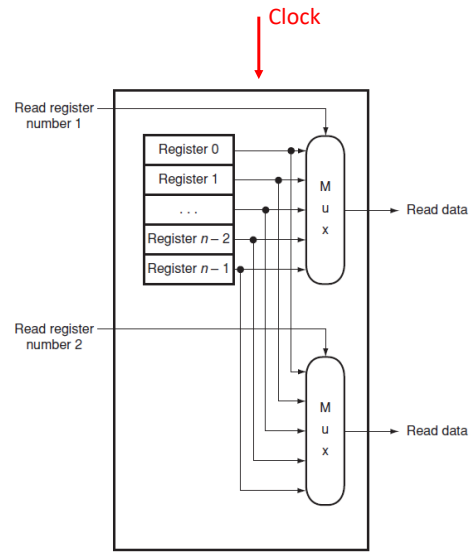



# MIPS R-format Instructions

- Περιλαμβάνει: **`add, sub, and, or, slt`**
- Βήματα:
  1. Διαβάζει 2 καταχωρητές (input)
  2. Εκτελεί μια αριθμητική ή λογική εντολή
  3. Γράφει το αποτέλεσμα πίσω σε ένα καταχωρητή (output)
- Οι 32 καταχωρητές γενικής χρήσης του επεξεργαστή αποθηκεύονται στο αρχείο καταχωρητών (**Register File - RF**)
  - Το RF είναι ένα σύνολο από 32 32-bit καταχωρητές όπου ο κάθε ένας μπορεί να διαβαστεί ή/και να γραφεί χρησιμοποιώντας τον αντίστοιχο αριθμό του καταχωρητή
  - Ανά πάσα στιγμή το RF περιέχει την κατάσταση των καταχωρητών (Register State) της μηχανής για την εκάστοτε εφαρμογή που εκτελείτε

# Register File (RF)



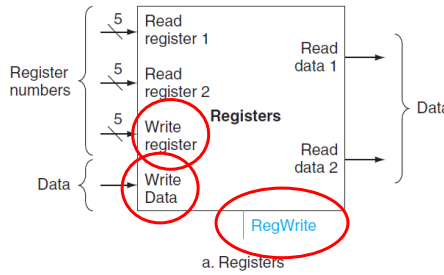





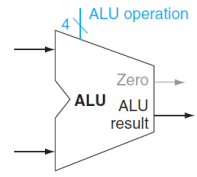





# MIPS R-format Instructions




a. Registers



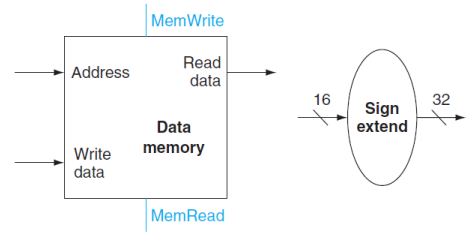

b. ALU

| ALU control lines | Function |
|---|---|
| 0000 | AND |
| 0001 | OR |
| 0010 | add |
| 0110 | subtract |
| 0111 | set on less than |
| 1100 | NOR |

- Απαραίτητες μονάδες για εκτέλεση: Register File και ALU
- Γιατί έχουμε 4 bit για το σήμα επιλογής του ALU?
- Μπορούμε να διαβάσουμε και να γράψουμε στον ίδιο καταχωρητή κατά την διάρκεια ενός μόνο κύκλου?
  - ΝΑΙ, αφού όλα παίρνουν ένα κύκλο και γράφω μόνο όταν γίνει edge-triggered (το CLK από 0 σε 1) και εφόσον έχω και τον Write Register, τα δεδομένα που θα γράψω και το RegWrite σήμα.
  - Αλλά, όταν γράφω, τα δεδομένα που διαβάζονται από τον ίδιο καταχωρητή είναι τα προηγούμενα

# MIPS Load/Store Instructions

- **lw $t1, offset_value($t2)**
- **sw $t1, offset_value($t2)**
- Βήματα:
  1. Διαβάζουμε τους register operands
  2. Υπολογίζουμε τη διεύθυνση χρησιμοποιώντας 16-bit offset (I-format εντολές)
     1. Χρησιμοποιούμε την ALU για την πράξη (**$t2+offset_value**), με sign-extend offset
  3. Load: Διαβάζουμε τα δεδομένα από τη μνήμη και τα γράφουμε στον καταχωρητή
  4. Store: Γράφουμε τα δεδομένα που κρατά ο καταχωρητής στη μνήμη
- Απαραίτητες μονάδες για εκτέλεση:
  - Μνήμη, Sign-extension Unit, Register File και ALU




a. Data memory unit b. Sign-extension unit



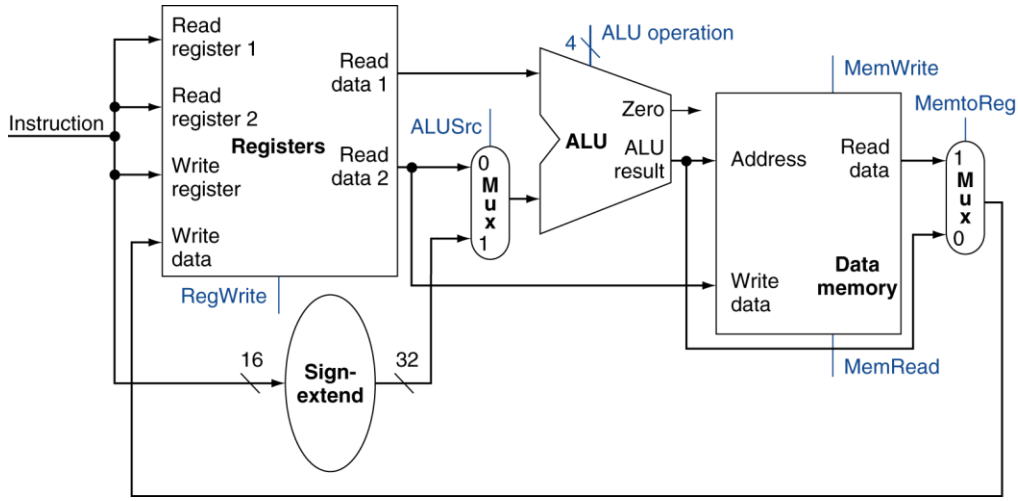




# Sign-extend offset

- Χρησιμοποιείται κυρίως για εντολές που έχουν να κάνουν με διευθύνσεις (I Format)
- Το immediate της εντολής συνήθως είναι 16 bit αλλά η διεύθυνση πρέπει να είναι 32 bit
- Με το Sign-extend αυξάνουμε τον αριθμό των bits σε 32 διατηρώντας τον αριθμό που αντιπροσωπεύουν τα 16 bits
  - Παίρνουμε το MSB (sign bit) και το τοποθετούμε σε όλες τις υπόλοιπες θέσεις μπροστά
  - Π.χ.
    - 16-bit offset: 1111111110110011 (decimal: -77)
    - 32-bit sign-extended number: 11111111111111111111111110110011 (decimal: -77)

# R-Type/Load/Store Datapath



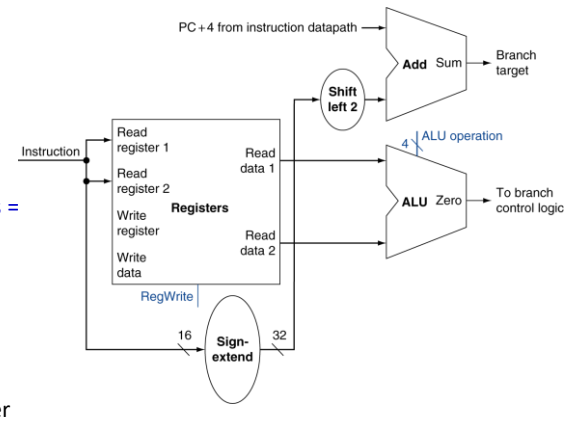







# MIPS Branch Instructions

- **beq $t1, $t2, branch_offset**
- Βήματα:
  1. Διαβάζουμε τους register operands
  2. Κάνουμε τη σύγκριση
     1. Χρησιμοποιούμε την ALU για την σύγκριση
     2. Αφαιρούμε και ελέγχουμε αν το αποτέλεσμα είναι 0 (τότε είναι ίσα)
  3. Υπολογίζουμε την διεύθυνση (target address = PC+4 + branch_offset*4)
     1. Sign-extend offset
     2. Shift left 2 places (byte addressable instructions → word addressable memory)
     3. Προσθέτουμε στο PC+4
        1. Υπολογιστικέ ήδη με το Instruction Fetch
- Απαραίτητες μονάδες για εκτέλεση:
  - Sign-extension Unit, Register File, ALU και Adder



# Συνδυάζοντας τα όλα…

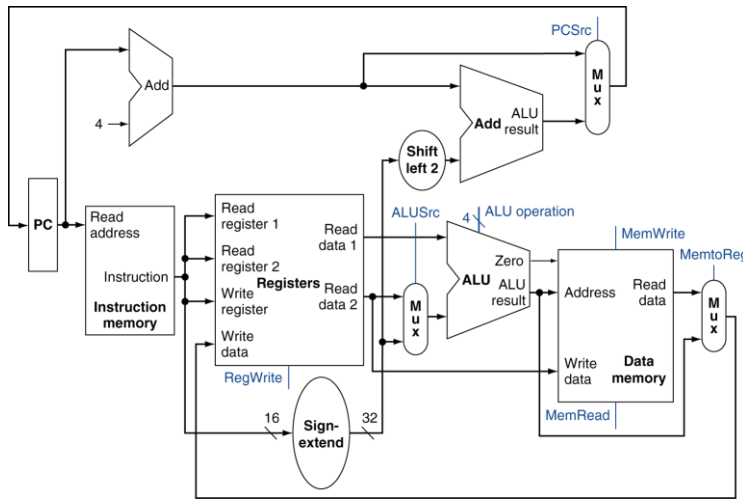
- Χρήση Multiplexers όταν έχουμε διαφορετικές πηγές δεδομένων αναλόγως της εντολής
- Σε αυτό το σενάριο, κάθε εντολή παίρνει 1 κύκλο για να εκτελεστή
  - Συνεπώς,
  - Κάθε μονάδα στο datapath μπορεί να κάνει 1 μόνο πράξη ανά κύκλο
  - Συνεπώς,
  - Χρειαζόμαστε διαφορετικές μονάδες μνήμης για τις εντολές και τα δεδομένα





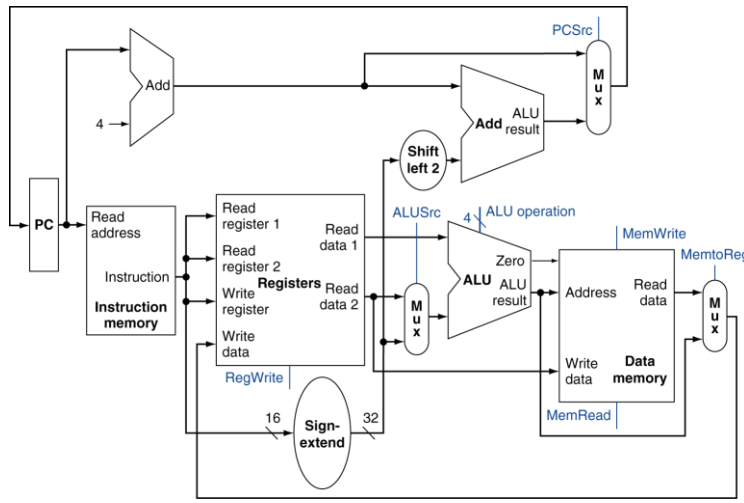


# Full Datapath



29

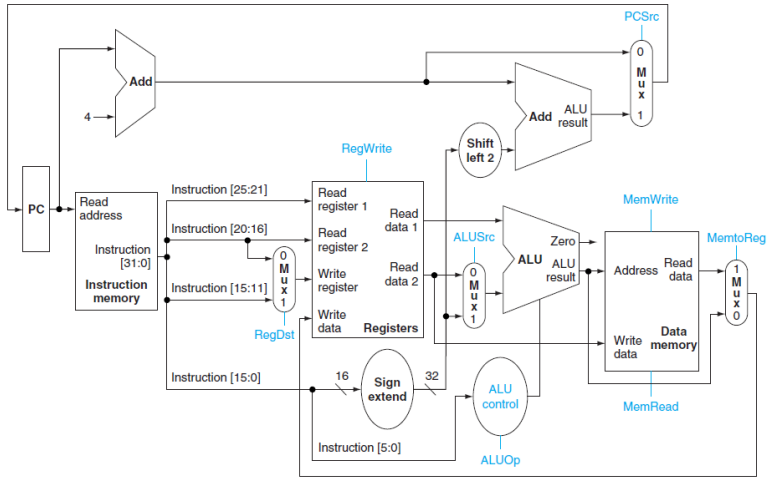
# Full Datapath






30

# Full Datapath with Control Signals







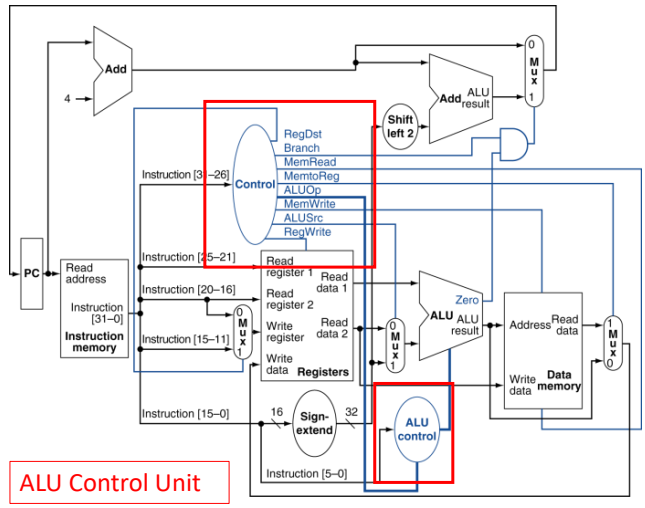



# Υλοποίηση Κεντρικής Μονάδας Ελέγχου

- Χρειαζόμαστε μια μονάδα ελέγχου για να συντονίζει και να συγχρονίζει την εκτέλεση των διάφορων εντολών
- Η κεντρική μονάδα ελέγχου (Control Unit) χρησιμοποιείται για:
  - Επιλογή των μονάδων που θα εκτελεστούν αναλόγως της εντολής (ALU, RF, etc.)
  - Έλεγχο της ροής των δεδομένων
    - Ανάθεση τιμών στα σήματα επιλογής των Multiplexers

# Datapath With Control Unit

Control Unit



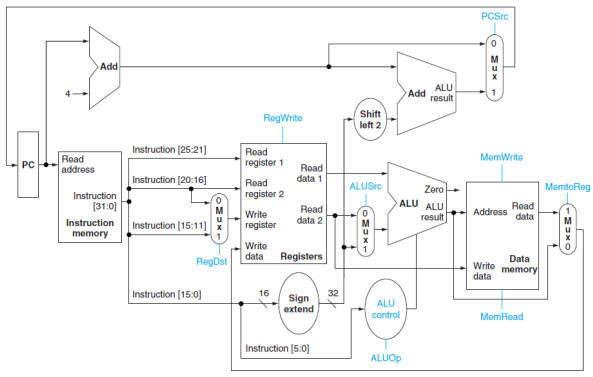

ALU Control Unit

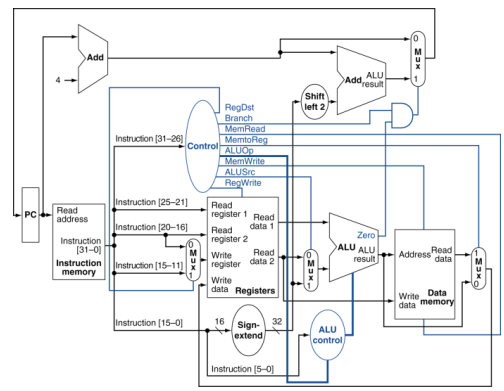






# Κεντρική Μονάδα Ελέγχου

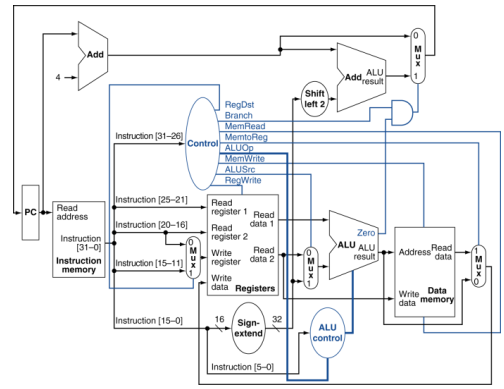



**Σήματα Ελέγχου** **Κεντρική Μονάδα Ελέγχου**

# Κεντρική Μονάδα Ελέγχου

| Όνομα Σήματος | Αποτέλεσμα όταν Απενεργοποιείται | Αποτέλεσμα όταν Ενεργοποιείται |
|---|---|---|
| RegDst | Καταχωρητής γραφής= `rt` | Καταχωρητής γραφής= `rd` |
| RegWrite | Κανένα | Καταχωρητής γραφής ανανεώνεται με δεδομένα γραφής |
| ALUSrc | 2ο όρισμα του ALU από Read data2 | 2ο όρισμα του ALU από sing-extended offset |
| PCSrc/Branch | PC=PC+4 | PC=PC + 4 + (4 x offset) |
| MemRead | Κανένα | Ανάγνωση από μνήμη δεδομένων |
| MemWrite | Κανένα | Γραφή στη μνήμη δεδομένων |
| MemtoReg | Δεδομένα για καταχωρητή γραφής από ALU | Δεδομένα για καταχωρητή γραφής από μνήμη δεδομένων |



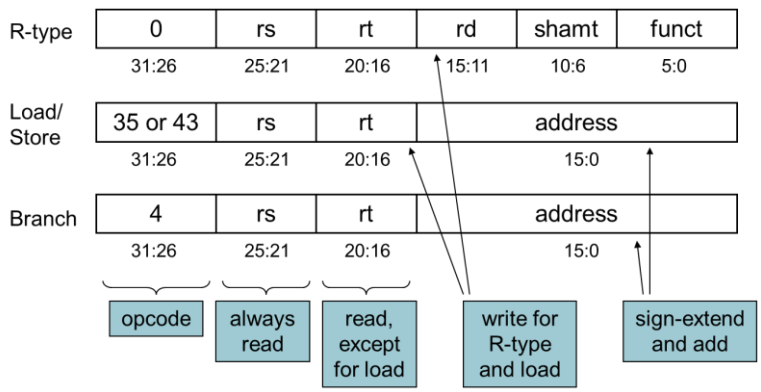

| Instruction | RegDst | ALUSrc | Memto-Reg | Reg Write | Mem Read | Mem Write | Branch | ALUOp1 | ALUOp0 |
|---|---|---|---|---|---|---|---|---|---|
| R-format | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| `lw` | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |
| `sw` | X | 1 | X | 0 | 0 | 1 | 0 | 0 | 0 |
| `beq` | X | 0 | X | 0 | 0 | 0 | 1 | 0 | 1 |



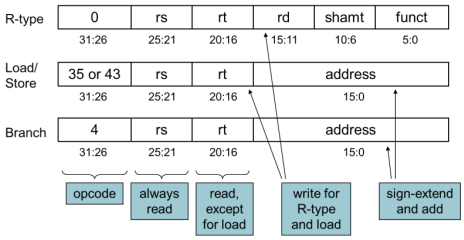


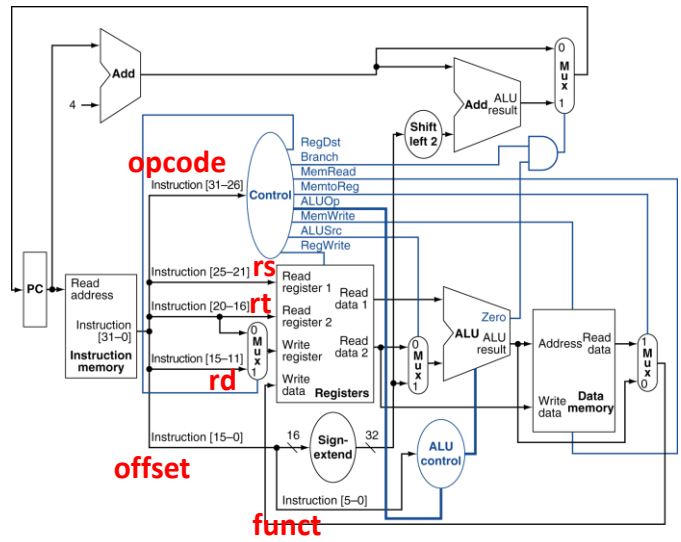


# Τα σήματα ελέγχου βγαίνουν από τις εντολές



# Κεντρική Μονάδα Ελέγχου










# Υλοποίηση Μονάδας Ελέγχου ALU

- Διαφορετική από την κεντρική μονάδα ελέγχου, αλλά εξίσου σημαντική γιατί
  - Σχεδόν όλες οι εντολές χρησιμοποιούν την ALU
- Η μονάδα ελέγχου ALU χρησιμοποιείται για:
  - Να ξέρει η ALU την πράξη που θα εκτελέσει αναλόγως της εντολής
- Η πληροφορία έρχεται από τα 32 bits της εντολής
- Παράδειγμα:
  - `add $8, $17, $18`
- Διάταξη εντολής:

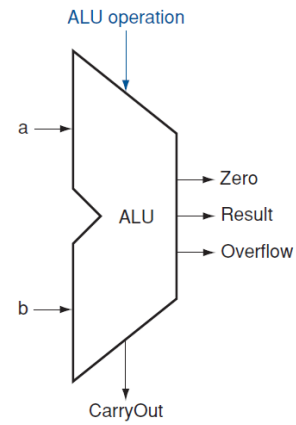
| 000000 | 10001 | 10010 | 01000 | 00000 | 100000 |
|---|---|---|---|---|---|
| op | rs | rt | rd | shamt | funct |

**R-type**

- Η λειτουργία της ALU εξαρτάται από τον τύπο της εντολής (opcode):
  - `op= 000000 = arithmetic operation`
- Και την πράξη που δίνεται (function code):
  - `funct= 100000 = add operation`

# Υλοποίηση Μονάδας Ελέγχου ALU

- Η ALU χρησιμοποιείται για:
  - Load/Store: Function = add
  - Branch: Function = subtract
  - R-type: Function = εξαρτάται από το πεδίο funct της εντολής




| ALU control lines | Function |
|---|---|
| 0000 | AND |
| 0001 | OR |
| 0010 | add |
| 0110 | subtract |
| 0111 | set on less than |
| 1100 | NOR |







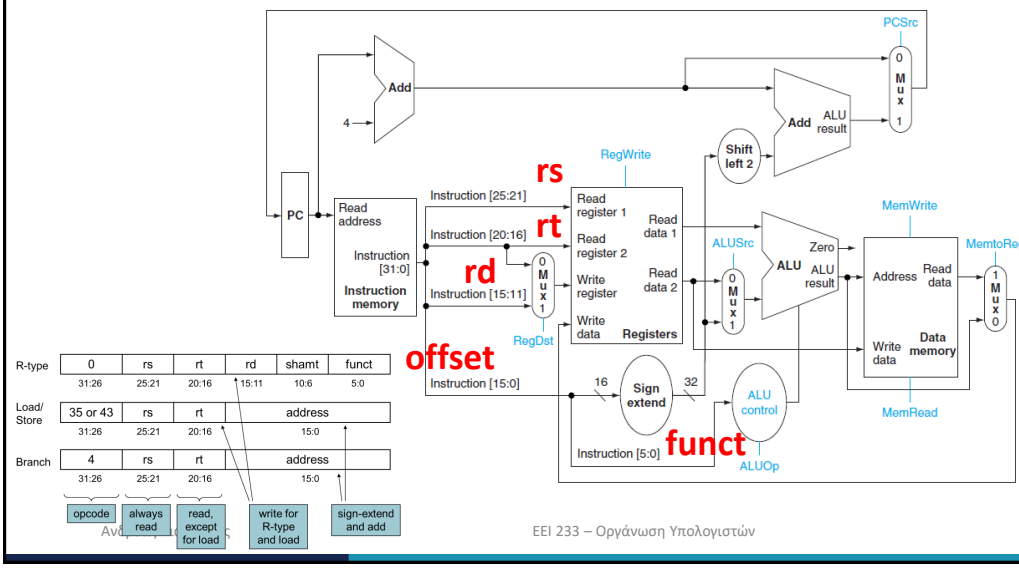
# Υλοποίηση Μονάδας Ελέγχου ALU

Σήμα από το Control Unit στο ALU Control

Σήμα από το ALU Control στην ALU

| Instruction opcode | ALUOp | Instruction operation | Funct field | Desired ALU action | ALU control input |
|---|---|---|---|---|---|
| LW | 00 | load word | XXXXXX | add | 0010 |
| SW | 00 | store word | XXXXXX | add | 0010 |
| Branch equal | 01 | branch equal | XXXXXX | subtract | 0110 |
| R-type | 10 | add | 100000 | add | 0010 |
| R-type | 10 | subtract | 100010 | subtract | 0110 |
| R-type | 10 | AND | 100100 | and | 0000 |
| R-type | 10 | OR | 100101 | or | 0001 |
| R-type | 10 | set on less than | 101010 | set on less than | 0111 |

# Ενσωμάτωση Μονάδας Ελέγχου ALU







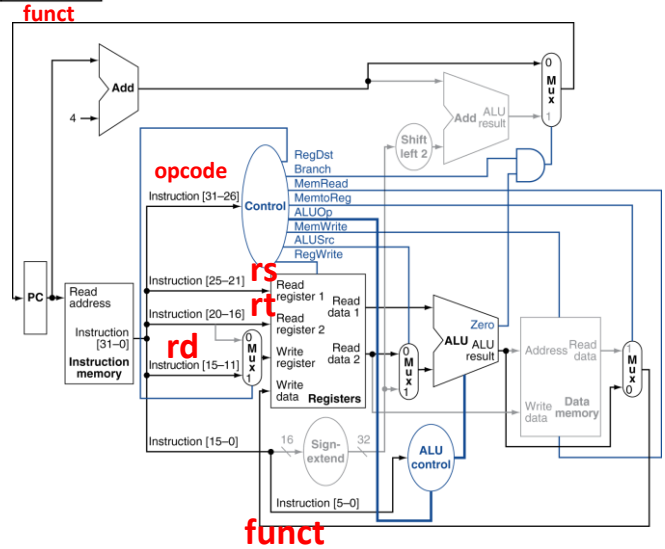

| 31:26 | 25:21 | 20:16 | 15:11 | 10:6 | 5:0 |
|---|---|---|---|---|---|
| 0 | $t2 | $t3 | $t1 | 0 | add |
| opcode | rs | rt | rd | | funct |

## R-Type Instruction

Παράδειγμα: `add $t1, $t2, $t3`

**Βήματα:**

1. Προσκόμιση εντολής και αύξηση του PC
2. Ανάγνωση καταχωρητών από το RF:
   1. `$t2 (25:21)`
   2. `$t3 (20:16)`
3. Το ALU ενεργοποιείτε βάση του funct
   1. Προσθέτει Read data 1 και Read data 2
4. Το αποτέλεσμα του ALU γράφεται πίσω στο RF
   1. `$t1 (15:11)`



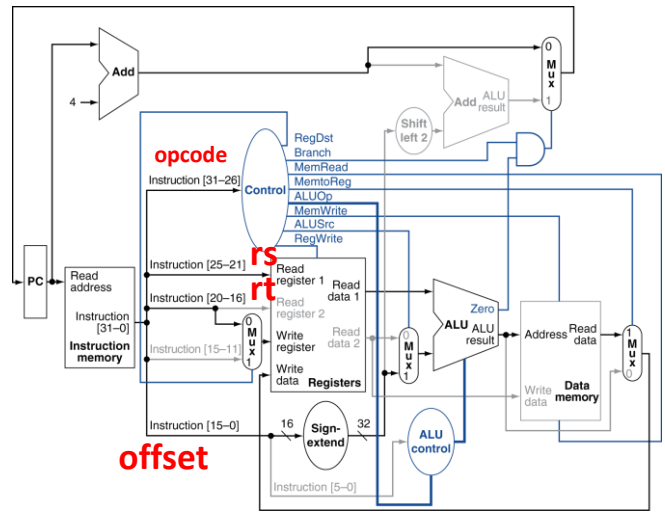
| 31:26 | 25:21 | 20:16 | 15:0 |
|---|---|---|---|
| 35 | $t2 | $t1 | offset |
| opcode | rs | rt | |

# Load Instruction

Παράδειγμα: `lw $t1, offset($t2)`

**Βήματα:**

1. Προσκόμιση εντολής και αύξηση του PC
2. Ανάγνωση καταχωρητών από το RF:
   1. `$t2(25:21)`
3. Το ALU ενεργοποιείτε βάση του ALUOp
   1. Προσθέτει Read data 1 και `offset(15:0)`
4. Το αποτέλεσμα του ALU είναι η διεύθυνση των δεδομένων στη μνήμη
5. Η έξοδος της μνήμης δεδομένων γράφεται στο RF
   1. `$t1(20:16)`







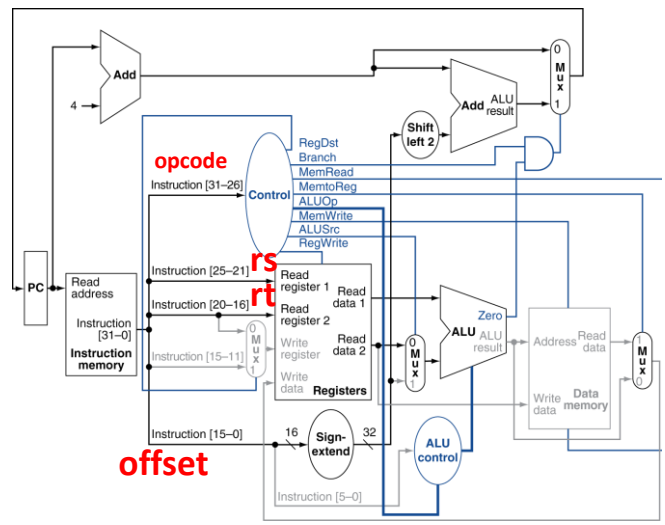

| 31:26 | 25:21 | 20:16 | 15:0 |
|---|---|---|---|
| 4 | $t1 | $t2 | offset |
| opcode | rs | rt | |

# Branch Instruction

Παράδειγμα: `beq $t1, $t2, offset`

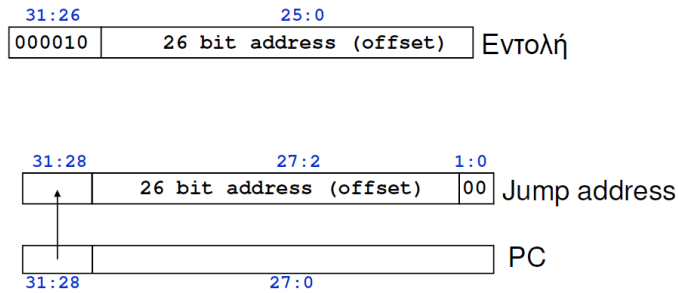
**Βήματα:**

1. Προσκόμιση εντολής και αύξηση του PC
2. Ανάγνωση καταχωρητών από το RF:
   1. `$t2(25:21)`
   2. `$t3(20:16)`
3. Το ALU ενεργοποιείται βάση του ALUOp
   1. Αφαιρεί Read data 2 από Read data 2
   2. Προσθέτει το sign-extend offset
   3. Shift Left 2 στο PC+4 για να υπολογίσει το branch target address
4. Ενεργοποιεί το ALU zero (= 1) για να αποθηκευτεί το branch target address



# Υλοποίηση Jump

- Παρόμοιο με το Branch αλλά πιο απλό!
- Jump address = 4 MSB του (PC+4) && offset*4

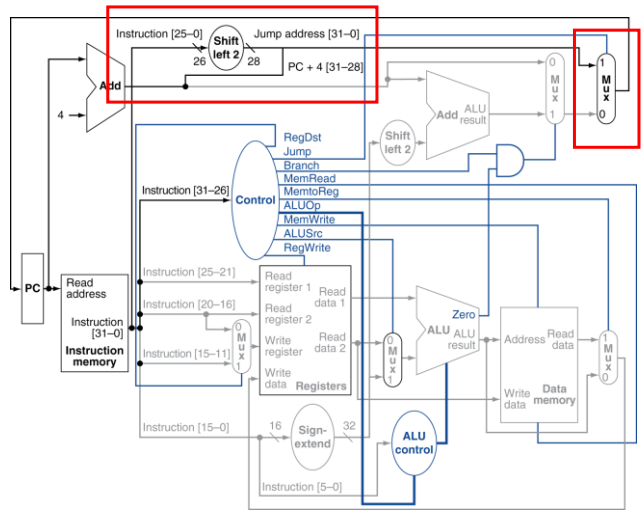






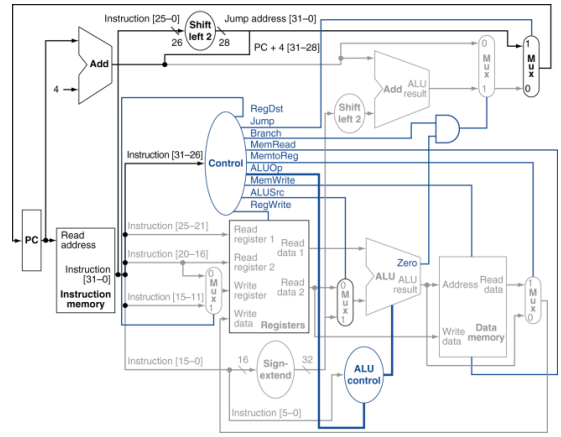



# Υλοποίηση Datapath Jump



# Μεθοδολογία ενός κύκλου ρολογιού

- Σημαίνει ότι κάθε εντολή παίρνει ακριβώς 1 κύκλο ρολογιού για να εκτελεστεί
- Πόσος πρέπει να είναι ο χρόνος ενός κύκλου ρολογιού?
  - Χρόνος για Jump?
  - Χρόνος για R-type?
  - Χρόνος για Load?
  - Χρόνος για Store?
- Ο χρόνος που πραγματικά χρειάζεται η κάθε εντολή καθορίζεται από τις μονάδες που θα χρησιμοποιήσει και πόσο χρόνο παίρνει η κάθε μία από αυτές
  - Όσες μονάδες εκτελούνται παράλληλα ο χρόνος τους κρύβεται!



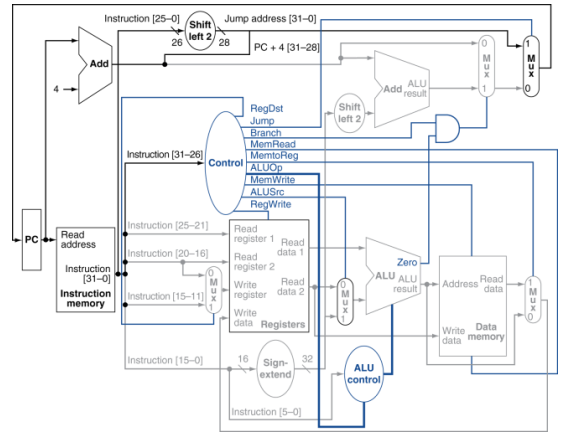







# Παράδειγμα

- Υποθέστε τον εξής χρόνο εκτέλεσης για κάθε μονάδα:
  - Memory units (Instruction και Data Memory): 200 picoseconds (ps)
  - ALU και adders: 100 ps
  - Register file (read or write): 50 ps
  - Όλα τα άλλα: 0 ps
- Πόσο χρόνο παίρνει μια JUMP εντολή?
  - 200 ps
- Πόσο χρόνο παίρνει μια ADD εντολή?
  - 400 ps
- Πόσο χρόνο παίρνει μια LOAD εντολή?
  - 600 ps
- Πόσο χρόνο παίρνει μια STORE εντολή?
  - 550 ps



# Παράδειγμα

- Υποθέστε τον εξής χρόνο εκτέλεσης για κάθε μονάδα:
  - Memory units (Instruction και Data Memory): 200 picoseconds (ps)
  - ALU και adders: 100 ps
  - Register file (read or write): 50 ps
  - Όλα τα άλλα: 0 ps
- Πόσο χρόνο παίρνει μια Jump εντολή?
  - 200 ps
- Πόσο χρόνο παίρνει μια ADD εντολή?
  - 400 ps
- Πόσο χρόνο παίρνει μια Load εντολή?
  - 600 ps
- Πόσο χρόνο παίρνει μια Load εντολή?
  - 550 ps

| Instruction class | Functional units used by the instruction class | | | | |
|---|---|---|---|---|---|
| R-type | Instruction fetch | Register access | ALU | Register access | |
| Load word | Instruction fetch | Register access | ALU | Memory access | Register access |
| Store word | Instruction fetch | Register access | ALU | Memory access | |
| Branch | Instruction fetch | Register access | ALU | | |
| Jump | Instruction fetch | | | | |

| Instruction class | Instruction memory | Register read | ALU operation | Data memory | Register write | Total |
|---|---|---|---|---|---|---|
| R-type | 200 | 50 | 100 | 0 | 50 | 400 ps |
| Load word | 200 | 50 | 100 | 200 | 50 | 600 ps |
| Store word | 200 | 50 | 100 | 200 | | 550 ps |
| Branch | 200 | 50 | 100 | 0 | | 350 ps |
| Jump | 200 | | | | | 200 ps |

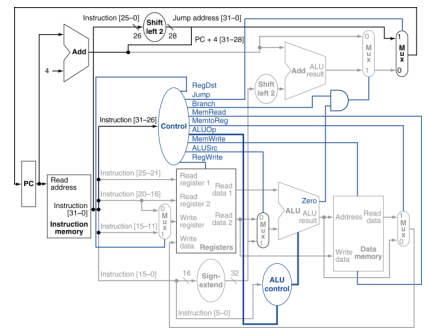






# Μεθοδολογία ενός κύκλου ρολογιού

- Πόσος πρέπει να είναι ο χρόνος ενός κύκλου ρολογιού?
  - Πρέπει να είναι η χειρότερη περίπτωση
  - **Ο χρόνος που παίρνει η πιο αργή εντολή!!**
- ΔΕΝ χρησιμοποιείται σήμερα γιατί δεν είναι η πιο αποδοτική λύση
  - Σπατάλη χρόνου
  - Μονάδες ανενεργές – δεν κάνουν τίποτα για μεγάλα χρονικά διαστήματα




| Instruction class | Instruction memory | Register read | ALU operation | Data memory | Register write | Total |
|---|---|---|---|---|---|---|
| R-type | 200 | 50 | 100 | 0 | 50 | 400 ps |
| Load word | 200 | 50 | 100 | 200 | 50 | 600 ps |
| Store word | 200 | 50 | 100 | 200 | | 550 ps |
| Branch | 200 | 50 | 100 | 0 | | 350 ps |
| Jump | 200 | | | | | 200 ps |

# Μεθοδολογία πολλαπλών κύκλων ρολογιού

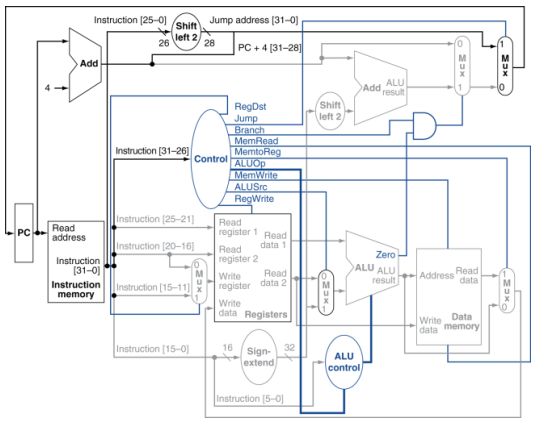
- **Ιδανικό σενάριο:** Κάθε εντολή να εκτελείτε σε 1 κύκλο, όπου η περίοδος ενός κύκλος ρολογιού είναι μεταβαλλόμενη αναλόγως της εντολής
  - Πολύ δύσκολο, έως αδύνατο, να υλοποιηθεί σωστά
- **Συμβιβασμός:** Σταθερή περίοδος για 1 κύκλο, αλλά κάθε εντολή μπορεί να εκτελεστεί σε περισσότερους ή λιγότερους κύκλους ρολογιού, αναλόγως του τύπου της εντολής
  - Σπάζουμε το Datapath σε μικρότερα κομμάτια τα οποία το κάθε ένα παίρνει 1 κύκλο ρολογιού







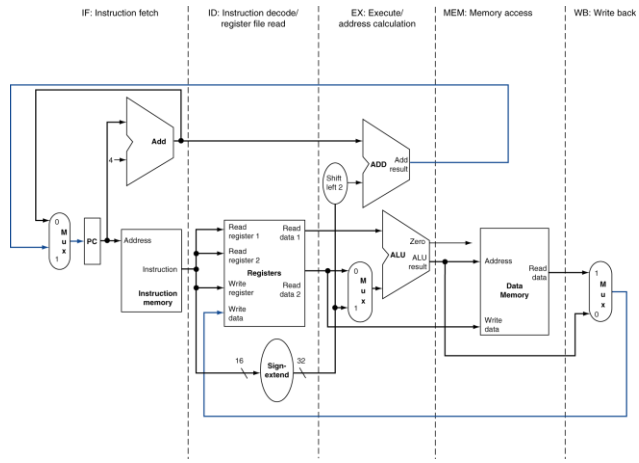
# Πως θα το σπάζατε?




Ένας κύκλος ρολογιού πρέπει να ισούται με τον χείριστο/μεγαλύτερο χρόνο εκτέλεσης οποιασδήποτε μονάδας του επεξεργαστή

| Instruction class | Instruction memory | Register read | ALU operation | Data memory | Register write | Total |
|---|---|---|---|---|---|---|
| R-type | 200 | 50 | 100 | 0 | 50 | 400 ps |
| Load word | 200 | 50 | 100 | 200 | 50 | 600 ps |
| Store word | 200 | 50 | 100 | 200 | | 550 ps |
| Branch | 200 | 50 | 100 | 0 | | 350 ps |
| Jump | 200 | | | | | 200 ps |

# Πως το σπάζει η MIPS



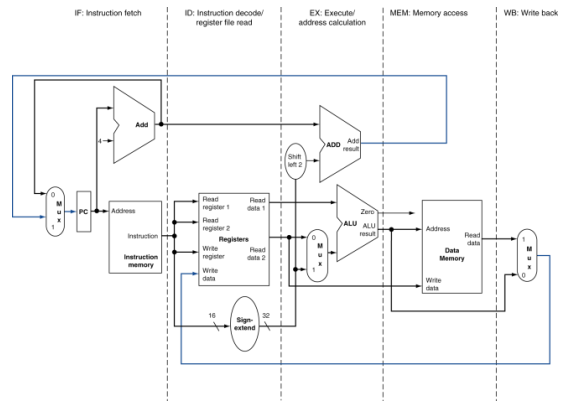







# Τα 5 στάδια του Datapath της MIPS

1. Φέρνουμε την εντολή από τη μνήμη
2. Αποκωδικοποίηση της εντολής και διάβασμα καταχωρητών
3. Εκτέλεση πράξης ή υπολογισμός διεύθυνσης στην ALU
4. Πρόσβαση στη μνήμη
5. Αποτέλεσμα πίσω στον καταχωρητή



# Τα 5 στάδια του Datapath της MIPS

- Πόσο χρόνο παίρνει η κάθε εντολή τώρα?
  - Όλες οι εντολές εκτελούνται μεταξύ 3-5 κύκλων
    - Κύκλους με μικρότερη περίοδο!
  - Τα στάδια 1 και 2 εκτελούνται για όλες τις εντολές
- **Πλεονεκτήματα:**
  - Μειώνουμε το χρόνο που κάποιες μονάδες είναι ανενεργές, αφού μπορούμε να έχουμε διαφορετικές εντολές μέσα στο Datapath στον ίδιο κύκλο
  - Μειώνουμε την συνολική σπατάλη χρόνου
- Αυτή η τεχνική ονομάζεται επίσης Pipeline (Διασωλήνωση)
  - Περισσότερα στην επόμενη ενότητα

# **Εργαστήριο:**
# Παραδείγματα εκτέλεσης εντολών μέσα στο Datapath





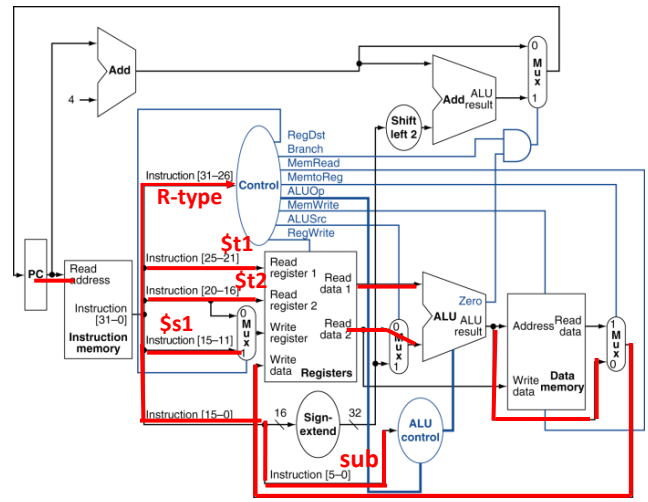


## Παραδείγματα

- `sub $s1, $t1, $t2`
- `lw $s1, offset($s2)`
- `beq $s2, $s4, offset`
- `j Loop1`

# sub $s1, $t1, $t2

| R-type | $t1 | $t2 | $s1 | 0 | sub |
|---|---|---|---|---|---|
| opcode | rs | rt | rd | | funct |




**Μονάδες που χρησιμοποιούνται:**

1. Instruction Memory
2. Register File
3. ALU

**Μονάδες που ΔΕΝ χρησιμοποιούνται:**

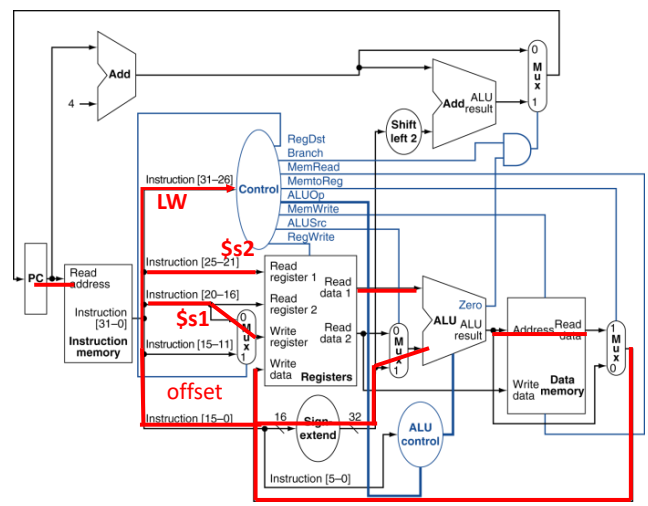
1. Branch Adder
2. Data Memory





# lw $s1, offset($s2)

| LW | $s2 | $s1 | offset |
|---|---|---|---|
| opcode | rs | rt | |




**Μονάδες που χρησιμοποιούνται:**

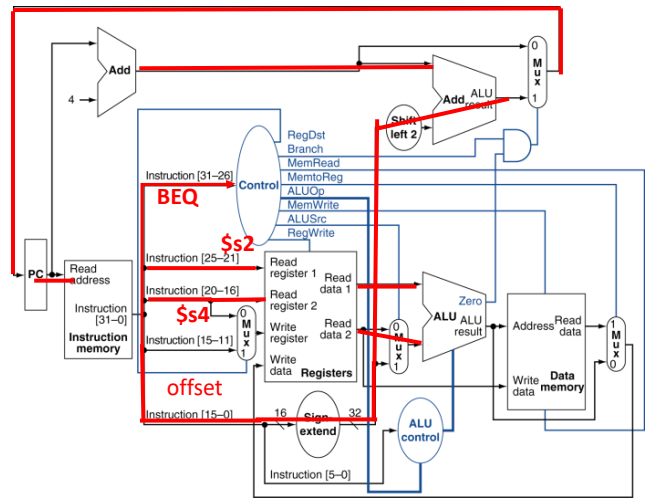
1. Instruction Memory
2. Register File
3. ALU
4. Data Memory

**Μονάδες που ΔΕΝ χρησιμοποιούνται:**

1. Branch Adder

# beq $s2, $s4, offset

| BEQ | $s2 | $s4 | offset |
|---|---|---|---|
| opcode | rs | rt | |




**Μονάδες που χρησιμοποιούνται:**

1. Instruction Memory
2. Register File
3. ALU
4. Branch Adder

**Μονάδες που ΔΕΝ χρησιμοποιούνται:**

1. Data Memory

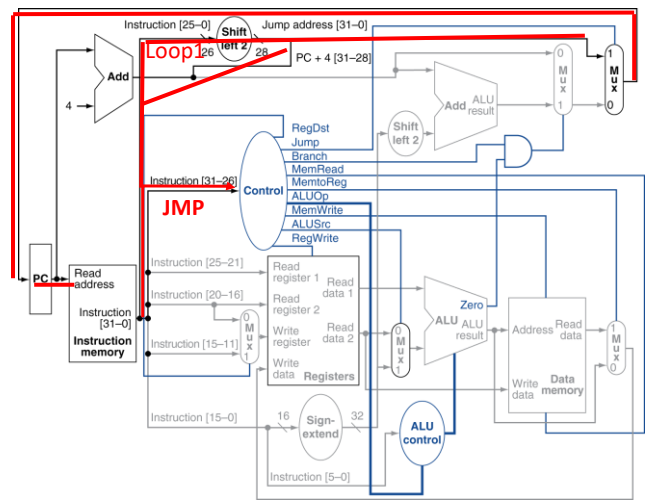


61



# j Loop1

| JMP | Loop1 |
|---|---|
| opcode | |




**Μονάδες που χρησιμοποιούνται:**

1. Instruction Memory

**Μονάδες που ΔΕΝ χρησιμοποιούνται:**

1. Data Memory
2. Register File
3. ALU
4. Branch Adder



62

# Ενότητα 05:
# The Processor:
# Datapath and Control

## ΕΕΙ 233: ΟΡΓΑΝΩΣΗ ΥΠΟΛΟΓΙΣΤΩΝ

**ΑΝΔΡΕΑΣ ΔΙΑΒΑΣΤΟΣ**

andreas.diavastos@cut.ac.cy

Τεχνολογικό Πανεπιστήμιο Κύπρου
Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

CARL

63